

Power of Two:-

16

$$1 \times 2 = 2 \times 2 = 4 \times 2 = 8 \times 2 = 16$$

$$\frac{16}{2} = \frac{8}{2} = \frac{4}{2} = \frac{2}{2} = 1$$

$$5/2 = 2$$

$$5.0/2 = 2.5$$

$$5/2.0 = 2.5$$

$$5.0/2.0 = 2.5$$

int n = 2.5;

n = 2

boolean isPowerofTwo (double n)

{

if (n == 1)

return true;

else if (n > 1)

return isPowerofTwo(n/2.0);

}

```
StringBuilder word = new StringBuilder("a");
```

```
void getword(int k) 
```

```
{
```

```
    if (k <= 1)
```

```
        return;
```

```
    else
```

```
    { StringBuilder temp = new StringBuilder();
```

```
        int len = word.length(); int i = 0;
```

```
        while (i < len)
```

```
        { if (word.charAt(i) == 'z')
```

```
            temp.append('a');
```

```
            else
```

```
            temp.append((char)(word.charAt(i) + 1));
```

```
            i++; k--;
```

```
        }
```

```
        word.append(temp);
```

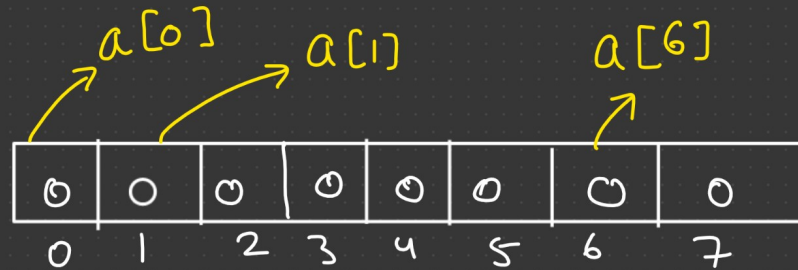
```
        getword(k);
```

```
}
```

Array:-

Array is collection of similar type of data element.

1D Array:-



`int a[];` → Declare
`a = new int[10];` → Assign



`int a[] = new int[10];`
initialization

Default value is zero.

`a[0] = 1;`

`a[1] = 4;`

`a[2] = 6;`

```
int a[] = { 2, 4, 0, 6, 7, 5, 4 };
```

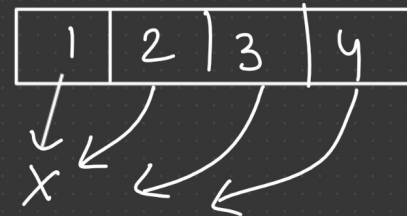
2	4	0	6	7	5	4
0	1	2	3	4	5	6

①

```
for( int i=0; i < a.length; i++)  
{  
    System.out.println( a[i]);  
}
```

②

```
for ( int x : a )  
{  
    System.out.println(x);  
}
```



Remove duplicate element:-

```
if (a[i] != a[i-1])  
{  
    a[k] = a[i];  
    k++;  
}  
i++;
```

$i = 1, 2, 3, 4, 5, 6, 7, 8$

	1	2	3						
0	1	2	3	4	5	6	7	8	

Diagram illustrating the removal of duplicate elements from an array. The array contains elements [0, 1, 2, 3, 4, 5, 6, 7, 8]. The first four elements (0, 1, 2, 3) are marked with 'X' above them, indicating they are being compared or processed. An arrow points from the 'X' above the element 4 to the variable 'k' below the array, indicating the position where the next unique element should be stored.

Search element in sorted array:-

$T=7$

1	2	4	6	8	10	15
0	1	2	3	4	5	6



$$\text{mid} = \frac{\text{left} + \text{right}}{2}$$

$$\text{mid} = \frac{0 + 6}{2} = 3$$

$$\begin{aligned} l &= 0 \\ r &= 6 \end{aligned}$$

$$m = \frac{0 + 6}{2} = 3$$

$$l = 4$$

$$r = 6$$

$$m = \frac{4 + 6}{2} = 5$$

return 4,

```
int left = 0; right = a.length - 1;
```

```
while ( left <= right)
```

```
{  
    int mid = (left + right) / 2;
```

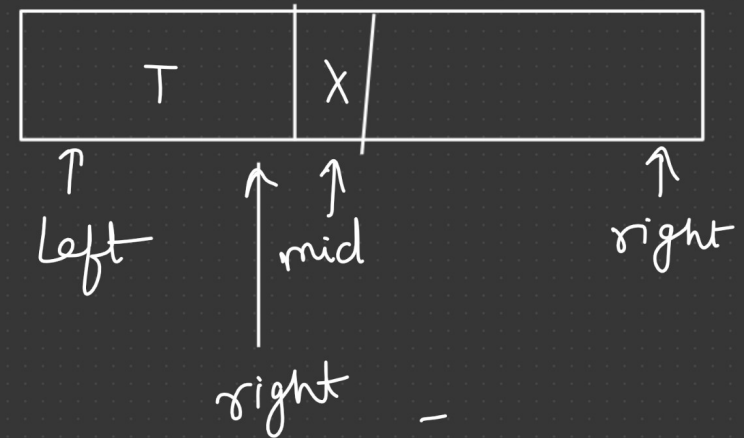
```
    if ( a[mid] == target)  
        return mid;
```

```
    else if ( a[mid] < target)
```

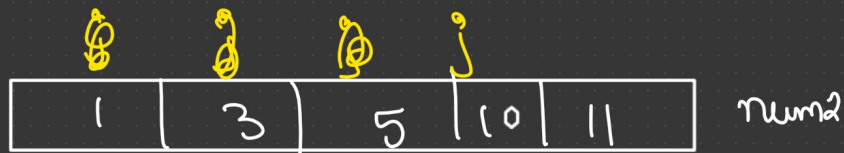
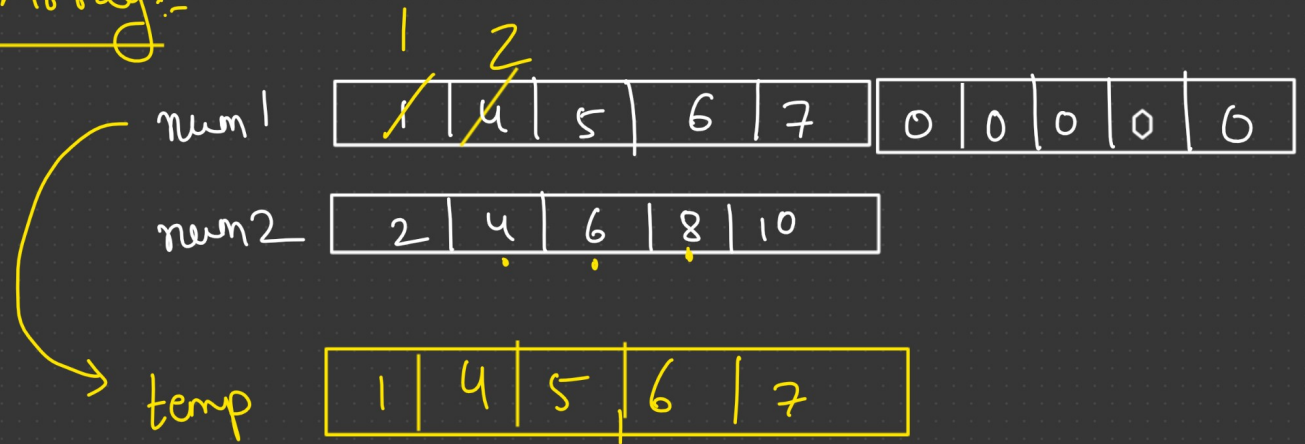
```
        left = mid + 1;
```

```
    else  
        right = mid - 1;
```

```
}  
return left;
```

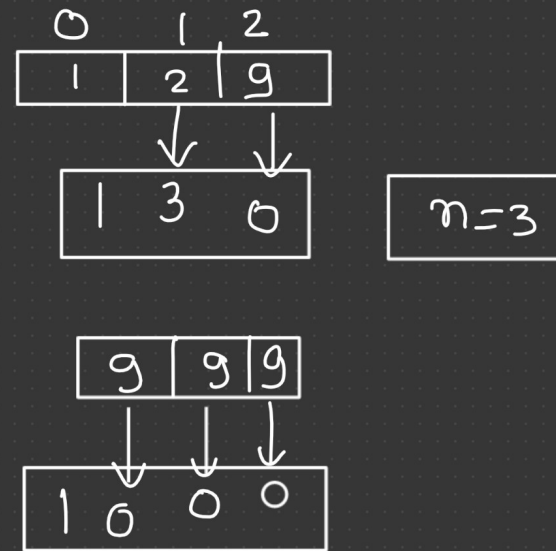


Merge Sorted Array:-



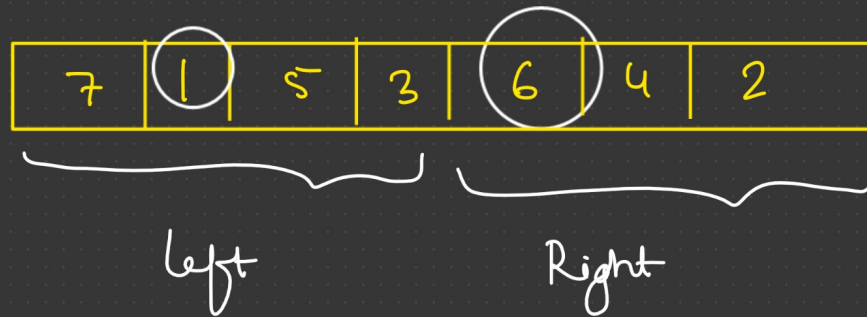
Plus One:-

```
for (int i = n-1; i >= 0; i--)  
{  
    if (digit[i] < 9)  
    {  
        digit[i]++;  
        return digit; ← X  
    }  
    digit[i] = 0;  
}
```

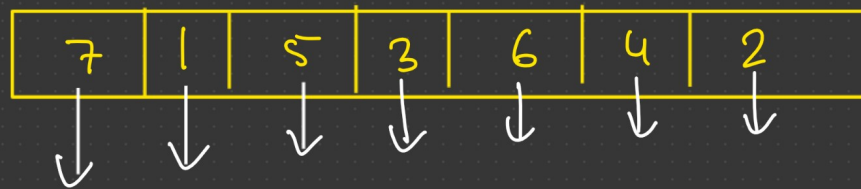


```
{  
    int temp[] = new int[n+1];  
    temp[0] = 1;  
    return temp;  
}
```

max profit in Stock:-



①
return max { left, right, $6 - 1 = 5$ }



min = ~~7~~ 1

profit = $5 - 1 = 4$
 $3 - 1 = \cancel{2}$
 $6 - 1 = 5$
 $4 - 1 = \cancel{3}$
 $2 - 1 = \cancel{1}$


```
int profit = 0; int buy = Integer.MAX_VALUE;
```

```
for (int x : prices )  
{
```

```
    if ( x < buy )
```

```
    {  
        buy = x;
```

```
    }
```

```
    else {
```

```
        int t = x - buy;
```

```
        if ( t > profit ) profit = t;
```

```
    }
```

```
return profit;
```

2-D Array in Java :-

	0	1	2	3	4
0	1	2	3	4	5
1	6	7	8	9	10
2	11	12	13	14	15
3	16	17	18	19	20

← A[0] → 1D Array

← A[1]

← A[2]

← A[3]

$A[2][3] = 14$
↓ ↓
row col.

Building [3][0]

1D Array

element

list

3	0	1	2	4
	0	1	2	3
	0	1	2	2
	0	1	2	1
	0	1	2	0

	0	1	2	3
0	00	01	02	03
1	10	11	50	13
2	20	21	22	23

```
int a[][] = new int[3][4];
```

$a[1][2] = 50$

3 one D - Array

4 elements in each 1D array

→ create 2D array

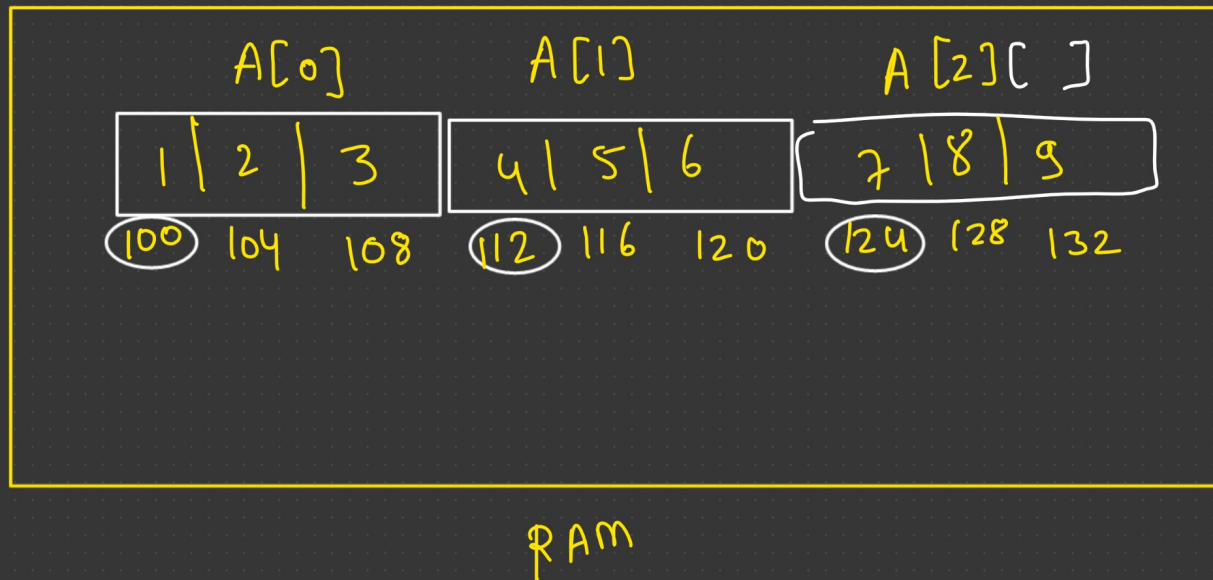
Total = $3 \times 4 = 12$

```

for (int i=0; i < a.length; i++)
{
    for (int j=0; a[i].length > j; j++)
    {
        a[i][j] = 2;
    }
}

```

→ no. rows
 → no. of elements
 $O(n^2)$



3D - Array

$A[0][1] \rightarrow$

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9
3	10	11	12

$A[0]$

	0	1	2
0	13	14	15
1	16	17	18
2	19	20	21
3	22	23	24

$A[1]$

$\leftarrow A[1][3]$

$A[1][3][2] = 24$

`int a[][][] = new int[2][4][3];`

No. of 2D Array

No. of rows
in each 2D
array

No. of elements
in each row

$$\text{Total} = 2 \times 4 \times 3 = 24$$

```

for (int i = 0; i < a.length; i++)
{
    for (int j = 0; j < a[i].length; j++)
    {
        for (int k = 0; k < a[i][j].length; k++)
        {
            a[i][j][k] = 2;
        }
    }
}

```

→ No. of 2D
array

→ No. of rows
in each
2D Array

→ No. of elements
in each row

$O(n^3)$

majority element:-

$$m = \times 2$$

$$\text{count} = \times 1$$

①

1	2	1	2	3	5	5	1
---	---	---	---	---	---	---	---

hash

0	1	1	1	0	1
0	1	2	3	4	5

$O(n)$

1 $\rightarrow$ 3

2 $\rightarrow$ 2

3 $\rightarrow$ 1

5 $\rightarrow$ 2

②

sorting:-

1, 1, 1, 2, 2, 3, 5, 5

$O(n \log n)$

1	1	2	1	3	1	1
--------------	--------------	--------------	--------------	--------------	--------------	---

1	1	2	2	2	3	2
--------------	--------------	--------------	--------------	--------------	--------------	--------------

result = ~~1~~ 2 ✓

Count = ~~1~~ ~~2~~ ~~1~~ ~~0~~ ~~1~~ ~~2~~ ~~1~~ 2

result = 1

Count = ~~1~~ ~~2~~ ~~1~~ ~~2~~

3 ~~2~~ ~~1~~